答辩材料 · 打印合订本

智维Agent

面向中小企业的可信 AI 智能运维与知识协同平台

SmartOpsAgent · 可信闭环 · 算法创新赛

赛事：2026 第八届全球校园人工智能算法精英大赛 · 河南赛区 · 算法创新赛

团队：**【团队名称 / 学校 / 队员 / 指导教师 —— 待填】**

版本：v2.0 · 2026-11

打印与装订顺序（从上到下）

1. **封面**（本页）
2. **目录**
3. **册一 · 陈述稿** —— 上台照着讲（建议单独打印一份拿在手里）
4. **册二 · 答辩全景手册** —— 主问答，评委深挖用
5. **册三 · 问答圣经** —— 快查补充
6. **册四 · 答辩补强** —— 临场技术追问 / 产品优势 / 未来前景
7. **附 · 一页速记卡** —— 建议彩打、裁下带上台

建议：A4 双面；打印时勾选"页眉页脚/页码"；速记卡单独一张纸。

目录

封面	封面（含打印顺序说明）	·····	1
目录	目录	·····	2
册一	陈述稿 · 上台陈述	·····	3 起
册二	答辩全景手册 · 主问答 70 题	·····	按分页
册三	问答圣经 · 速查补充 40 题	·····	按分页
册四	答辩补强 · 技术追问 60 题 / 产品优势 / 未来前景	·····	按分页
附	一页速记卡（可裁下带上台）	·····	末页

册一

陈述稿 · 上台陈述（约 8–10 分钟）

源文件：defense.md

智维Agent 项目竞赛答辩稿（夺奖版 v2.0）

> 项目名称：**智维Agent —— 面向中小企业的可信 AI 智能运维与知识协同平台**
> 参赛赛事：**2026 第八届全球校园人工智能算法精英大赛 · 河南赛区 · 算法创新赛**
> 建议时长：**陈述 8–10 分钟 + 问答 5–7 分钟**（按秩序册时间微调）
> 配套文档：`innovation.md`（创新点）、`project-plan.md`（计划书）、`demo-script.md`（演示脚本）、`defense-qa.md`（**问答圣经 35 题**）
> 文档版本：v2.0（在 v1.0 基础上重构叙事、强化算法创新、补齐评分映射与防坑指南）

0. 上台前 30 秒：一句让人记住的话

> **"企业不敢把生产环境，交给一个只会回答问题的 AI。"**
> 这句话，就是我们做智维Agent 的全部理由。

（停顿，看评委，再开口。）

1. 开场与痛点（约 70 秒）

各位评委老师好。我们是 **【团队名称 / 学校待填】** 队，参赛作品是 **智维Agent —— 面向中小企业的可信 AI 智能运维与知识协同平台**。

先讲一个真实场景：一家 50 人的公司，半夜两点数据库告警。值班的是刚入职三个月的应届生，他打开搜索框，搜出来的排障文档，是**去年旧版本**的；他问大模型，大模型给了个**听起来很有道理、却没法验证**的方案；他照着一做——高风险命令直接执行，**没有任何人复核、没有任何记录**。

这不是科幻，这是中小企业的日常。它们的运维有三个共性约束：**人手少、系统杂、知识散**。由此带来四个具体痛点：

1. **知识是旧版本的**——检索返回过期文档，越搜越错；
2. **结论是黑盒的**——大模型说了算，运维不敢照做；
3. **执行是没有边界的**——AI 能分析，却不该能直接动生产；
4. **过程是留不住的**——处理完就散了，无法复盘、无法沉淀。

所以我们的判断很明确：**企业真正需要的，不是更会聊天的机器人，而是一个"结论有证据、动作有边界、过程有留痕"的 AI 同事。**

2. 解决方案：给 AI 装"三道锁 + 一本账"（约 90 秒）

智维Agent 不是一个单点的 AI 调用，而是一套**可信闭环**。我们把一次运维任务拆成可运行的 eleven 步：

监测 → 分析 → 知识检索 → 证据验证 → 决策规划
→ 风险判断 → 人工审批 → 安全执行 → 结果验证 → 审计留痕 → 经验沉淀

用一句人话概括：**AI 负责思考，人负责授权，系统负责留痕。**

工程上，它由四端协同构成：

- **前端** Vue 3 + Vite，用户层 / 管理员层双入口；
- **后端** Spring Boot，管账户、权限、审计、附件会话；
- **AI 服务** FastAPI，管模型网关、对话编排、RAG；
- **Python Agent 框架（可插拔算法内核）**，五个核心模块：planner（规划）/ executor（执行）/ inspector（核查）/ memory（记忆）/ policies（策略），经 **82 个单测**验证；产品级运行闭环由后端 `DecisionLoopService` 承载。

但比"能跑起来"更重要的是——我们给这套系统装了**三道锁和一本账**：

- **第一道锁：证据锁。** 没有证据支撑的结论，不得表述为确定事实。
- **第二道锁：风险锁。** AI 可以规划，但高风险动作必须人工审批，绝不允许自作主张。
- **第三道锁：边界锁。** 工具在沙箱与权限注册表里运行，能力被严格收口。
- **一本账：审计账。** 每一次判断与动作，都结构化留痕，可回溯、可复盘。

这就是智维Agent 与所有"套壳聊天机器人"的本质区别。

---

3. 现场演示（约 100–120 秒，衔接 demo-script.md）

> 演示只走两条主线，不铺页面。提前录好 60 秒兜底视频（见第 11 节）。

主线一：一次带证据的故障问答。
我用语音说"MySQL 连接数打满"，附一张现场截图。请大家看屏幕左侧——AI 的"思考过程"被拆成六个阶段逐条点亮：**图像证据解析 → 知识检索 → 证据验证 → 任务规划 → 风险判定 → 生成回答**。回答右侧会列出它**引用了哪些来源、命中多少条证据、可信度评分（trust_score）是多少**。

这里我要特别强调一个隐私取舍：**语音在浏览器内识别、音频不出端；截图只传文件名、图片内容不上传。** 我们演示多模态，但不以泄露用户隐私为代价。

主线二：一个被"拦下来"的高危动作。
Agent 拆解出一个"重启核心服务"的步骤。策略引擎立刻判定为**高风险**，自动转入人工审批队列——未经批准，它一步也动不了。我点击批准后，后端决策环在**沙箱**中执行该动作，并用我们自研的**四级纠察（规则 → 证据 → AI → 状态核验）**做结果核验——其中"假成功"检测会拿"声称已解决"对比"真实系统状态"，一票否决；最后在审计与运行记录里，你能看到一条完整的、带风险等级与审批人的条目。

这两条线合起来，就是我们要证明的事：**AI 再强，也只在人允许的范围内、在可核查的边界内，可信地工作。**

---

4. 算法与技术深度（约 90 秒）—— *本节是算法创新赛的得分核心*

评委老师最关心的，一定是"你们的算法到底深在哪"。我讲三个我们真正做了算法设计的点。

① 版本感知的混合检索 + 可信度评分算法。
通用 RAG 只解决"召回"，但中小企业最常见的故障不是"查不到"，而是"查到的是旧版本"。我们的做法是分层算法链：

- **召回层**：BM25 关键词召回与向量召回**并行**，避免单路径漏召回；
- **过滤层**：`version_filter` 按目标环境版本过滤候选，把"知识正确但版本不对"的噪音挡在门外；
- **收敛层**：词法重排器（`lexical_reranker`）对候选重新排序；
- **表达层**：检索结果聚合为 **Citation（来源 / chunk / 版本 / 分数）**，并进一步聚合为 `trust_score` **可信度评分。**

把"版本"做成检索的**一等公民**，这是与通用 RAG 的实质差异，也是我们的核心算法贡献之一。

② 多级证据验证算法（inspector 四级纠察）。
结果核查不是"比对字符串"，而是**规则 → 证据 → AI → 状态核验**四级融合：规则层做硬约束（风险误判、审批绕过、循环、无证据先动手），证据层做一致性校验（版本匹配、结论是否有据），AI 层做语义补充判定，最后由**状态核验层拿"Agent 声称已解决"对比"真实系统状态"，一票否决"假成功"（false success）**。其中前三级与状态核验都不依赖模型，保证安全判定不被一个"自信的模型"带偏。

③ 风险策略引擎。
`policies` 基于权限注册表与风险注册表，对每一步动作打风险等级，高危动作默认拒绝或转人工。它把"安全"从一句口号，变成插在每一步执行**之前**的可运行代码路径。

> 一句话：别人在"调提示词"，我们在"设计算法 + 工程化约束"。

---

5. 创新点（约 60 秒，按重要性讲 3 点）

1. **可信闭环工程化**（最重要）。把权限、风险、审批、核查、审计做成**可运行的代码路径**，而非文档理念。这是从"AI 对话"到"AI 运维"的关键一跃。
2. **版本感知检索**。解决"知识正确但版本不对"这一行业通病，把版本做成检索一等公民。
3. **透明推理交互**。让用户**看见** AI 检索了什么、命中多少证据、判了什么风险——直接提升信任度。

补充创新（一句带过）：多模态隐私边界分层、插件化工具生态（Docker/K8s/MySQL/Nginx/Redis + MCP）、可切换模型网关（默认 mock 零密钥可复现）、双入口同源前端工程化。

---

6. 工程质量与可复现性（约 40 秒）

一个能拿奖的作品，必须经得起"能不能跑起来"的检验：

- **前端**：175 个 .vue 文件，用户层 / 管理员层**双入口构建通过**（注：`npm test` 494 项全过，另经 `tsc --noEmit` 与双入口构建验证）；
- **后端**：**671 个 @Test（126 个测试类）**，基于 H2（MySQL 兼容模式）跑真实迁移脚本；
- **数据库**：MySQL 8，**119 张表**，迁移脚本幂等，启动自动补齐；
- **可复现**：`mock` 模型**零密钥**即可跑通全链路，一键切换到 Ollama / OpenAI 兼容模型。评委老师可以自己拉下来跑。

---

7. 诚实边界：我们如何对待"还没跑出来的指标"（约 30 秒）—— *防坑必讲*

我必须坦诚一件事：当前仓库里**模型训练与基准评测尚未接入真实模型与标注数据集**，所以我没有拿"准确率 99%"这样的数字来包装自己。

但我把这当成一种**工程成熟度的体现**——我们专门搭建了**诚实的评测框架**：消融实验（RAG 链、Inspector 链）、基线对比链、leakage 检测、指标契约（任务成功率 / 根因准确率 / 证据准确率 / Inspector 精确率召回率 / 幻觉率 / 假成功率 / 安全违规率）全部设计完毕，**没有真实观测数据前，系统强制返回 `NOT RUN`，绝不编造数字。**

在算法创新赛里，我反而认为：**一个知道"哪些结论还不能下"的系统，比一个什么都敢编的系统更可信。** 这不是缺点，这是我们对"可信 AI"四个字的交代。

---

8. 社会价值与商业潜力（约 40 秒）

- **社会价值**：把大厂才有的运维体系能力，以低成本交付给中小企业与高校实验室，降低对少数"老师傅"的个人经验依赖。
- **商业潜力**：SaaS 订阅（按席位）+ 私有化部署双形态；知识库与插件作为增值模块；模型用量分成。目标客户是 10–200 人规模企业。
- **可推广性**：默认 mock 可离线演示，适合教学、实训与竞赛场景。

9. 收尾（约 20 秒）

回到开头那句话——

> 企业不敢把生产环境，交给一个只会回答问题的 AI。
> 智维Agent 想做的，是让每一位运维工程师，都拥有一个**可信、可控、可复盘**的 AI 同事。

我们的陈述到此结束，谢谢各位评委老师！

附录 A：评分维度映射（每讲一段，就在评委心里"打卡"）

评分维度	权重（参考）	本稿对应段落	你要在台上"亮"的点
AI 技术深度 / 算法	35%	第 4 节	版本感知混合检索、trust_score、四级证据核查（规则→证据→AI→状态核验）、风险引擎
创新性	25%	第 2、5 节	三道锁+一本账、可信闭环工程化、透明推理
前端工程质量	10%	第 6 节	610 个 Python 用例 + 671 个后端 @Test、双入口同源、分包优化
实用性	10%	第 3、8 节	真实运维闭环、mock 可复现、模型可切换
用户体验	10%	第 3 节	思考可视化、渐进增强、隐私提示
社会价值与商业潜力	10%	第 8 节	中小企业普惠、SaaS+私有化

> 提示：若秩序册公布的权重与上述不同，**第 4 节（算法）与第 5 节（创新）是无论如何都要讲满、讲透的两段。**

附录 B：30 秒电梯演讲（候场/被临时打断时用）

> 智维Agent 是一个可信 AI 智能运维平台。它用"版本感知混合检索 + 证据可信度评分 + 四级证据核查（规则→证据→AI→状态核验） + 风险策略 + 人工审批 + 全程审计"，把大模型从"会回答问题"变成"可信地完成运维任务"。四端全栈、610 个 Python 用例 + 671 个后端 @Test、零密钥可复现。一句话：AI 负责思考，人负责授权，系统负责留痕。

附录 C：演示时间盒（建议）

时间	内容
0:00–0:20	开场钩子 + 痛点
0:20–1:30	解决方案（三道锁+一本账）
1:30–3:30	Demo 主线一（带证据问答）
3:30–5:30	Demo 主线二（高危动作被拦截→审批→执行→审计）
5:30–7:00	算法深度 + 创新点
7:00–8:00	工程/诚实边界/价值/收尾

附录 D：答辩"三要三不要"

- **要** 每页 PPT 只讲一句结论，其余让图说话；
- **要** 被问边界时**先承认、再给路线图**，比硬撑更得分；
- **要** 结尾抛回那句"AI 负责思考，人负责授权，系统负责留痕"，让评委记住；

智维Agent 材料 · 仅评委可见

- **不要** 念稿、不要堆数字不讲含义、不要对没跑出的指标含糊其辞（直接按第 7 节话术）；
- **不要** 贬低 ChatGPT / LangChain（说"差异在约束，不在模型"）；
- **演示翻车** 立即切 60 秒备用视频，或展示 `technical-document.html` 的链路图，不要慌。

附录 E：待团队补齐的占位（提交前必填）

- 团队名称、学校、队员姓名/学号/专业、指导教师
- 若秩序册评分权重与本稿不同，请在第 4、5 节加权重示
- 在线演示地址（可选加分项）

册二

答辩全景手册·主问答（70 题）

源文件：defense-master.md

智维Agent 答辩全景手册（评委视角·最全·最通透）

> 适用：2026 第八届全球校园人工智能算法精英大赛 · 河南赛区 · 算法创新赛
> 用法：**陈述看 `defense.md`，问答看本手册。** 本手册按"评委为什么问 → 怎么答 → 代码证据 → 防坑"组织，背要点、不背全文。
> 原则：**结论先行·一句证据·一句边界/路线图。** 先承认，再超越。

0. 先想清楚：评委到底在考什么

评委不是来听你念 PPT 的，他在快速判断四件事：

1. **这是不是真东西**（有没有跑起来的系统，还是 PPT 工程）；
2. **算法到底深不深**（算法创新赛尤其看这点，不能只是"调了提示词"）；
3. **你懂不懂自己写的东西**（能不能追到代码、追到公式、追到边界）；
4. **有没有培养价值**（诚实、可复现、有路线图）。

所以你的每一句回答，都要能落回这四件事之一。**切忌：堆名词、绕边界、背稿感。**

1. 项目一页通（四个时长版本，按需取用）

30 秒（电梯稿）

> 智维Agent 是一个可信 AI 智能运维平台。它用"版本感知混合检索 + 证据可信度评分 + 四级纠察 + 风险策略 + 人工审批 + 全程审计"，把大模型从"会回答问题"变成"可信地完成运维任务"。四端全栈、610 个 Python 用例 + 671 个后端 @Test、零密钥可复现。一句话：AI 负责思考，人负责授权，系统负责留痕。

1 分钟

> 中小企业运维"人手少、系统杂、知识散"，最怕两件事：AI 给错建议没人发现，AI 做错动作没人拦得住。通用聊天机器人解决不了"可信"。我们做了智维Agent：一次运维任务被拆成"监测→分析→检索→证据验证→规划→风险判断→人工审批→安全执行→结果验证→审计→沉淀"的闭环。检索带版本过滤和可信度评分，执行带风险分级和人工审批，全程审计留痕。它有四端：Vue 前端、Spring Boot 后端、FastAPI AI 服务、Python Agent 框架。默认 mock 零密钥就能跑通。

3 分钟 = 1 分钟 + 现场走一遍"高危动作被拦截→审批→执行→核查→审计"的主线 + 点出三个算法：版本感知混合检索、trust_score、四级纠察。

核心记忆句

> **AI 负责思考，人负责授权，系统负责留痕。**
> 三道锁：证据锁 / 风险锁 / 边界锁；一本账：审计账。

2. 核心算法"通透"讲解（被深挖时用，这是算法创新赛的命门）

2.1 版本感知混合检索：到底怎么"混"的

真实实现（`knowledge/retrieval/hybrid.py`）：

- 向量检索与 BM25 检索**并行**跑，且都传入同一个 `version`；
- 两路结果**各自按本路最大分归一化**（`score / max_score`），消除量纲差异；
- 融合公式：`final = 0.6 × 向量归一化分 + 0.4 × BM25归一化分`；
- 按 `chunk_id` 合并去重、排序、取 `top_k`；
- 权重 0.6/0.4 是**可配置参数**，不是写死。

版本过滤（`version_filter.py`）：`version` 为空或 `"latest"` 时不过滤；否则**精确匹配** `result.version`。也就是说版本在**召回阶段就已下推**（`search(..., version=...)`），过滤是二次保险。

为什么这么设计（通透版）：

- 单路检索必漏：向量擅长语义、BM25 擅长精确术语（如 `max_connections`、错误码），运维日志两者都重要，所以并行召回；
- 归一化必须**按路做**：向量分是余弦相似度、BM25 分是无界词频分，直接相加会被 BM25 量级带偏；
- 版本下推到召回，避免"先召回一堆旧版本、再过滤到空"。

必背的"边界"（评委爱问）：精确匹配偏保守，版本号写法不一致会误杀——这是**刻意用精度换召回**，因为运维场景"宁缺毋滥"；升级路径是语义化版本区间 + 版本归一化。

2.2 trust_score：可信度到底怎么算的

真实公式（`knowledge/evidence/trust_score.py`）：

```
trust_score = clamp( 0.5 × mean(证据分) + 0.3 × [结论被支持] + 0.2 × min(证据数/3, 1), 0, 1 )
```

- `0.5`：证据本身的平均相关度（检索有多准）；
- `0.3`：结论是否被证据支持（二值，来自证据一致性判定）；
- `0.2`：证据覆盖度，3 条封顶（避免靠堆条数刷分）；
- 结果截断到 [0,1]，保留 4 位小数。

为什么这三项：一个可信的答案，必须"检索得准 + 结论有据 + 证据够量"，缺一不可。权重是**可调初始值**，后续用消融实验拟合。

通透点：`trust_score` 不是模型黑盒打分，它是**可逐条溯源**的——每条 Citation 带 `source / chunk_id / version / score`，评委问"0.82 怎么来的"，你能拆到每一项。

2.3 四级纠察：不是三级，是四级（这是加分点）

真实实现（`agent/inspector/pipeline.py`）：**Rule → Evidence → AI → Verifier** 四级，最后聚合成一个 `InspectionResult`。

级别	做什么	不依赖模型?
L1 Rule Judge	确定性规则：风险误判、审批绕过、工具循环（>10 次）、无证据就做状态变更	**是**（纯代码）
L2 Evidence Judge	证据充分性、版本匹配、结论与证据一致性（词面重合：≥0.5 支持 / ≥0.2 部分 / 否则不支持）	**是**（启发式）
L3 AI Judge	语义级综合判定，**可插拔** `judge_fn`，默认不绑定模型（passthrough）	否（可注入模型）
L4 State Verifier	拿"Agent 声称已解决"对比"真实系统状态"，抓 **false success**	**是**（规则）

聚合状态机（`_aggregate`）：

- 出现 `false_success` → **REJECTED**（一票否决，触发 Re-plan）；

- 最高严重级为 HIGH/CRITICAL → **VIOLATION**；
- 有其它 issue → **WARN**；否则 **PASS**；
- VIOLATION / REJECTED 或高危 → `requires_human = true`。

为什么这是"算法"而不只是"工程"：它把"结果可信"建模成一个**多信号融合 + 状态机聚合**的判定问题，核心是**假成功（false success）的检测**——Agent 说"已解决"、系统 502 还在，这一票否决是 LLM 自己永远给不出的。

2.4 风险策略与安全守卫：两道闸

风险分类（`agent/policies/risk.py`）：

- `CRITICAL`：`server.delete`、`database.drop`（禁止自动执行）；
- `HIGH`：`server.restart`、`server.stop`、`server.delete`、`shell.execute`（必须人工审批）；
- `MEDIUM`：其它 `server.*`；其余 `LOW`。

PolicyEngine：CRITICAL → 直接拒；HIGH 且未审批 → 拒并返回 `requires_approval=true`；其余放行。
SafetyGuard：权限不通过 → 拒；CRITICAL → 阻断；HIGH 且未审批 → 要求审批；否则放行。

通透点：为什么两道闸而不合并？**职责分离**——PolicyEngine 面向"计划步骤"做准入，SafetyGuard 面向"执行动作"做最后拦截，双保险防单点失效。这是安全工程里典型的 defense-in-depth。

2.5 透明推理（SSE）：哪些是真算的，哪些是呈现

真实实现（`ai-service/app/services/chat.py` 的 `chat_stream`）：产出三类事件 `thinking / token / done`；思考阶段 `vision(有图才有) → retrieve → evidence → plan → risk → answer`。

必须诚实的一点（防坑）：阶段里的 `detail` 用的是**真实数据**（图像张数、命中证据条数、top_k），但阶段推进有 0.15s 的节奏延时，`plan/risk` 在对话链里是**标签位**——真正的规划与风险判定在 `agent/` 层执行。**不要说"每个阶段都是后端实时算出来的"**，说"阶段由真实请求数据驱动，视觉节奏做了演示优化"。

2.6 Agent 内核：为什么自研

- `planner` 出带依赖的步骤；`kernel` 提供插件注册、依赖解析、事件总线、生命周期；`registry` 管 tool/skill/model/permission/risk 五类注册表；`sandbox` 管文件/进程/网络策略；`providers` 支持 mock/模型。
- **自研理由**：要的是"**策略与审批优先**"的执行模型，能把风险判定插在**每一步执行之前**；通用链式编排做不到这种"执行前拦截"。

---

3. 评委高频问题（按类分组 • 70 题）

> 标记：⭐=必答核心（要背熟）；🔥=高频陷阱（要练话术）；普通=流畅带过即可。

A. 项目与定位（8 题）

A1 ⭐ 一句话介绍你的项目？
骨架：是什么 + 解决什么 + 一句价值。作答见 §1 电梯稿。防坑：别超 40 秒。

A2 为什么做"可信运维"？
中小企业"人手少、系统杂、知识散"，最怕"AI 给错建议没人发现、AI 做错动作没人拦得住"。这是真实且未被满足的需求。

A3 你的用户和典型场景？
10–200 人企业运维负责人/驻场工程师；故障问答定位、版本感知检索、受控执行、核查沉淀、团队协同。

A4 完成度到哪？
四端全栈可运行；前端 `npm test` 494 项全过、后端 126 测试类、119 张表；mock 零密钥跑通。模型基准评测尚未接入（见 E 类）。

A5 和"运维知识库"差在哪？
知识库只解决"查得到"，我们解决"查得对版本 + 答得有证据 + 做得有边界 + 过程能复盘"。

有答辩问题，答辩有边界

A6 项目最大的技术难点?

把"可信"从口号变成**可运行、可拦截、可追溯**的代码路径——尤其是"假成功检测"和"风险执行前闸门"。

A7 如果重做，你会改什么?

先接真实模型与标注集，把消融实验跑通；检索的版本过滤从精确匹配升级到语义化版本区间。

A8 项目名字有什么含义?

"智维"= 智能运维；核心不是"更聪明的聊天"，而是"可信地完成运维"。

B. 算法与检索（12 题）—— 算法创新赛重点

B1 ⭐ 🔥 你的"算法创新"到底在哪?

三点：① 版本感知混合检索 + trust_score 聚合；② 四级纠察（含 false-success 状态机）；③ 风险分级→审批闸门的策略算法。**强调：不是调提示词，是可公式化、可溯源、可消融的设计。**

B2 ⭐ 混合检索怎么融合?

见 §2.1：并行召回 → 各自 max 归一化 → 0.6/0.4 加权 → 合并去重排序。**能报出权重**是加分。

B3 🔥 为什么不用 RRF?

RRF 是秩融合、鲁棒；我们用的是**归一化分加权**，可调、可解释、能按场景调权重。两者都合理，RRF 在消融计划内。**别贬低 RRF。**

B4 🔥 只用 max 归一化，outlier 会不会压扁其他分?

会，这是 max 归一化的已知局限；min-max 又对最小值敏感。当前选它是因为简单稳定，接口可换。承认局限 + 给替代。

B5 ⭐ trust_score 公式?

`0.5×证据均分 + 0.3×是否被支持 + 0.2×min(证据数/3,1)`，截断[0,1]。见 §2.2。**能背公式是"通透"的硬证据。**

B6 🔥 0.5/0.3/0.2 怎么定的? 凭什么?

初始启发式权重，代表"检索准 > 结论有据 > 证据量"的优先级；**是可调参数**，最终应由消融实验拟合。诚实说"经验初值 + 待数据标定"。

B7 向量嵌入用的什么?

默认轻量 `hash_embedding`（可复现、零依赖），可切换 Sentence Transformers。**承认 hash 不是语义嵌入**，它是演示默认，不是能力上限。

B8 🔥 版本过滤只是精确匹配，太粗暴?

是偏保守的精确匹配，`latest` 不过滤；优点是不放旧版本进来，缺点是同义版本号会误杀。**这是用精度换召回**，升级路径是语义化版本区间。

B9 检索效果怎么评?

`knowledge/evaluation` 有指标与样例集；但**真实基准未跑**（见 E 类）。可讲设计，不编数字。

B10 证据一致性怎么判的?

词面 token 重合率：≥0.5 SUPPORTED、≥0.2 PARTIALLY、否则 UNSUPPORTED（`evidence_judge.py`）。**这是确定性启发式**，好处是不会被"自信的错误答案"骗过；升级路径是语义模型/NLI。

B11 🔥 词重叠判一致性，语义相同但用词不同的情况怎么办?

承认：纯词面会漏"同义不同词"。所以它只是 L2 的确定性基线，L3 AI Judge 做语义补充；两者融合降低漏判。**这是分层设计的意义。**

B12 检索复杂度 / top_k 怎么定?

top_k 默认 5（可配）；两路各取 top_k 再融合，复杂度随候选线性；工程上够用，未做大规划压测。

C. Agent 与工程架构（10 题）

C1 ⭐ 是单次 LLM 调用吗?

不是。planner 拆步骤 → executor 调工具 → inspector 核查 → memory 沉淀，多模块协作。

C2 planner 怎么规划?

把故障拆成**带依赖关系的步骤**（`planner/steps.py`），每步绑定工具、风险、是否需审批。

C3 🔥 自研 Agent 框架图什么？不用 LangChain?

要"策略与审批优先"的执行模型，把风险判定插在**每步执行之前**；kernel 有注册表/事件总线/生命周期，通用链式编排做不到执行前拦截。

C4 插件化怎么做的?

`AgentPlugin` 契约注册；已内置 Docker/K8s/MySQL/Nginx/Redis；MCP 适配器接外部工具。新环境加插件不改主流程。

C5 多智能体体现在哪?

后端 `TeamAgentRunService` + `DecisionLoopService` 支撑团队级运行与决策循环；单机闭环 + 团队协同两层。

C6 memory 怎么设计?

短期上下文 + 长期 `operator_memory` 双层；处置结论回流，形成"越用越懂"的经验。

C7 🔥 119 张表是不是过度设计?

覆盖账户权限/审计/团队/知识/模型网关/哨兵/远程运维完整域；迁移幂等，局部缺失只影响对应模块。**承认"偏重"，给"模块化裁剪"路线。**

C8 性能/大屏卡不卡?

zrender 从 echarts 拆出（510→346kB），Three.js 懒加载，图表面板按需加载，管理员层独立产物。

C9 后端为什么用 Spring Boot，AI 用 FastAPI?

后端要成熟的权限/审计/事务；AI 侧要 Python 生态与异步流式。**职责分层**，不是堆技术。

C10 一次对话的完整数据流?

用户输入(文本/语音/图像文件名) → 前端 SSE 客户端 → ai-service：RAG 检索→证据→Prompt 组装 → LLM 网关 → SSE 回流(thinking/token/done) → 高风险：策略→审批→执行→核查→审计。

D. 安全 / 隐私 / 伦理（8 题）

D1 ⭐ 数据安全怎么保证?

BCrypt(12)、CSRF、会话固定防护、管理员 MFA+恢复码、AES-GCM 密钥加密、附件 nosniff+CSP sandbox、结构化审计。

D2 🔥 多模态是"假多模态"吗?

诚实答：图像当前**文件名级**参与链路，内容不上传（刻意隐私取舍）；语音浏览器内识别、音频不出端；OCR 模块已在，路线图是授权后接内容级。**不把"传文件名"包装成"看懂图片"。**

D3 密钥怎么管?

环境变量注入 + AES-GCM 加密存后端，绝不入库、不明文，`.env` gitignore。

D4 🔥 AI 自动执行会不会很危险?

三重约束：权限注册表 + 风险策略引擎 + 沙箱；高危必须审批；执行后核查+审计。**危险在架构层被卡死。**

D5 如果模型给出错误的高危建议?

提示词约束"无证据不下结论/高风险不声称已执行"；风险评估独立于模型；高危必须人工审批。**模型错了也不会直接成事故。**

D6 用户数据能不能用于训练?

默认不用于训练；隐私边界分层（音频不出端、图片不出端、附件隔离），符合最小必要原则。

D7 权限模型?

RBAC：角色/部门/岗位；权限注册表 + 允许角色集（admin/operator）；管理员操作二次确认。

D8 🔥 你不觉得"人工审批"很反效率吗?

效率与安全的权衡：低风险自动化、高风险人工。**审批只拦"不可逆/高影响"动作**，不是所有动作都拦。这是"可信自动化"而非"全自动"。

E. 评测与"无数据"陷阱（8 题）—— 必练

E1 ⭐ 🔥 你的准确率/效果数据是多少?

坦诚：当前未接入真实模型与标注数据集，没有编造的基准数字。 但搭了诚实评测框架：消融（RAG 6 组 / Inspector 5 组）+ 基线链 + leakage 检测 + 指标契约（任务成功率/根因/证据/工具/风险/审批/验证/Inspector P&R/幻觉率/假成功率/安全违规率）。无观测数据前系统强制 `NOT RUN`。**一个知道"哪些结论还不能下"的系统，比什么都敢编的更可信。**

E2 🔥 没数据凭什么说好?

两层：**系统能力**已可演示可验证（闭环/检索/审批/审计在跑）；**模型效果**需真实数据，如实标注未跑。竞赛看算法设计与工程实现，这两层拿得出手。

E3 消融做了吗?

设计完成（RAG 六组、Inspector 五组），**尚未执行**——缺真实模型/数据集/evaluator 三件套，门禁不允许空跑。

E4 🔥 为什么用 mock? 没接真模型?

mock 为**零密钥可复现**，非能力上限；`model_gateway` 支持 Ollama/OpenAI 兼容，运行期切换；后端管连接/路由/Token 预算/用量。

E5 你的可信度评分验证过吗?

公式可拆、可溯源；但**权重未用真实数据标定**，属于设计阶段。诚实。

E6 假成功率怎么定义?

Agent 声称"已解决/已恢复"，而实际状态 `resolved=false`（如仍 502）→ false success → REJECTED（`state_verifier.py`）。

E7 🔥 你怎么保证不是"自评自嗨"?

评测契约要求：统一分组测试集、留 artifact hash、缺失值记 `null` 而非 0、零分母不报 0。**这是防自嗨的机制。**

E8 如果现在就让你跑一个真实评测，缺什么?

真实模型提供方 + 已标注且通过泄漏检查的分组测试集 + evaluator 适配器。三者到位即可出数。

F. 工程与可复现（7 题）

F1 ⭐ 测试情况?

前端 494 项全过 + `tsc --noEmit` + 双构建；后端 126 个 JUnit 类（H2 MySQL 兼容跑真实迁移）；119 表迁移幂等。

F2 前端为什么"部分 TS"?

共享逻辑层（48 模块）迁 TS 拿类型收益，组件层留 JS 控成本；`tsc --noEmit` 已进门禁。**有意取舍，不是偷懒。**

F3 🔥 610 个 Python 用例 + 671 个后端 @Test是不是凑数?

覆盖契约、安全不变量、领域逻辑，不是纯断言堆砌；核心是"防单人阻塞"的交接保证。

F4 怎么保证可复现?

mock 零密钥跑通全链路；一键切 Ollama/OpenAI；docker-compose；迁移自动补表。

F5 CI/质量门禁?

`npm test / typecheck / build / build:admin` + `mvn test` + `pytest`，命令都在 `project-plan.md`。

F6 有没有性能数据?

分包做了针对性优化，但**未做系统压测**；如实说，不编 QPS。

F7 部署形态?

docker-compose 一键起；SaaS / 私有化双形态；生产需配 TLS 与共享限流。

G. 商业与落地（6 题）

G1 商业闭环?

SaaS 订阅(按席位) + 私有化授权 + 知识库/插件增值 + 模型用量分成；目标 10–200 人企业。

G2 中小企业凭什么买单?

零密钥可演示、可私有化自托管数据、降低对"老师傅"的依赖，成本远低于养完整运维团队。

G3 你的护城河?

不是模型（可换），是"可信约束闭环 + 领域知识资产 + 插件生态"的**系统性整合**。

G4 怎么获客?

高校实验室/实训机房切入（同源需求）→ 中小企业口碑 → 行业方案商渠道。

G5 成本结构?

模型用量是大头，默认可本地 Ollama 控成本；私有化一次性授权。

G6 下一步路线?

图像内容级理解（授权后）、端侧推理（Transformers.js/WebGPU）、多节点会话+MFA、团队协同深化。

H. 竞品与对比（6 题）

H1 🔥 和 ChatGPT + 提示词的区别?

差异**不在模型，在约束**：版本感知检索、证据可信度、风险分级审批、审计回流。模型可换，约束不换。ChatGPT 给答案，我们给"可负责任的运维动作"。

H2 和 Dify / Coze / LangGraph 比?

那些是通用编排平台；我们是"可信运维"垂直闭环——**风险闸门在每步执行前**、假成功检测、版本感知检索是它们的空白。

H3 和传统 AIOps?

AIOps 多停在规则/阈值告警，有日志无推理、有自动化缺审批；我们补"证据可信度+风险审批+透明推理+审计沉淀"。

H4 和 RAGFlow / FastGPT 比?

它们强在文档 RAG；我们强在"检索→证据→纠察→审批→审计"的**端到端可信链**，且把版本做进召回。

H5 你的差异化一句话?

别人让 AI 更会答，我们让 AI 更敢被信。

H6 🔥 这些能力大厂都有，你凭什么?

大厂能力不普惠、不轻量、不私有化；我们把"可信运维"做成中小企业可负担、可自托管、可离线演示的形态。

I. 团队与过程（5 题）

I1 团队几人怎么分工?

前端交互 / 后端数据 / AI 算法三块切分；测试与构建门禁防单人阻塞。

I2 你个人负责哪块?

（据实回答，落到具体模块与代码，别泛泛。）

I3 🔥 1–3 人做得完吗?

按模块切 + 门禁保交接 + 文档脚本保可复现；610 个 Python 用例 + 671 个后端 @Test就是"某人请假也不崩"的保证。

I4 最大的坑?

（讲真实的一个：如迁移漏表、双入口配置分裂，说清怎么发现、怎么修。）

I5 用了 AI 辅助写代码吗?

如实：用了 AI 辅助提效，但架构、算法、验收由人把关；关键路径有人审。**诚实反而加分。**

J. 压力 / 刁难 / 送命题（8 题）—— 练心态

J1 🔥 "我觉得你这项目没什么技术含量。"

理解您的视角。技术含量不在"训了个大模型"，而在**把可信约束工程化**：版本感知检索融合、trust_score 公式、四级纠察+假成功状态机、风险执行前闸门、可复现评测框架——每处都是针对"AI 运维可信"这个真问题的算法与系统设计。欢迎就任一模块深入切磋。

J2 🔥 "创新点别人都能做，凭什么获奖？"

单点别人或可做，但把"版本过滤+证据聚合+风险审批+透明推理+全程审计"**串成可运行、可演示、可复现的闭环**，每点都有代码与演示路径——**系统性整合与工程纪律本身就是壁垒。**

J3 🔥 "你这就是套壳吧？"

套壳是"换个皮调 API"。我们有自研 Agent 内核、检索/证据/纠察/风险四套算法、119 表域模型、610 个 Python 用例 + 671 个后端 @Test。**壳下面是自己的骨架。**

J4 🔥 "你连真实数据都没有，凭什么谈算法？"

算法价值在设计与可证伪性：公式可拆、状态机可测、消融可跑。我们**刻意不编数字**，这是对"可信"二字的自洽——一个敢说"我还不能下结论"的系统，才配谈可信。

J5 🔥 "现场能跑吗？跑挂了怎么办？"

能跑（mock 零密钥）。预案：60 秒备用录屏 → 技术文档链路图 → 口述主线+点代码位置。**不慌、不重来、不长停顿。**

J6 🔥 "你这个安全设计，真出事谁负责？"

高危动作**必须人工审批**才执行，审批人留痕；AI 只规划不越权。**责任边界在架构里就划清了**——这正是"可信"的核心。

J7 🔥 "如果评委只给你 1 分钟，讲什么？"

讲那句：企业不敢把生产环境交给只会回答的 AI；我们用"证据锁+风险锁+边界锁+审计账"，让它变成敢托付的 AI 同事。**一句话 + 三道锁。**

J8 🔥 "你觉得你最大的不足是什么？"

真实数据闭环还没跑通，这是最硬的短板；已把评测框架和门禁先建好，接上真实模型与标注集即可出数。**承认最硬的，别挑软的。**

K. 前沿与展望（4 题）

K1 如果接多模态大模型，架构要改吗？

不用大改：图像走"证据解析"入口，`ChatRequest.images` 已是扩展点，接视觉模型即可内容级理解。

K2 端侧推理可行吗？

可行，路线图有 Transformers.js/WebGPU；适合隐私敏感、离线场景。

K3 多智能体怎么深化？

团队级 `TeamAgentRunService/DecisionLoopService` 已有骨架，深化方向是角色分工+协商+共享记忆。

K4 你怎么看 AI 运维的未来？

从"辅助回答"走向"可信代理"：关键不是更聪明，而是**可授权、可拦截、可追溯。**我们押的是"信任基础设施"。

---

4. "不会答"的三段式救场（万能）

1. **接住**："这个问题很关键，我分两层说。"
2. **给已知**："就我目前实现/验证到的部分，是……（讲你确定的一点）。"
3. **划边界 + 路线**："更完整的结论需要……（真实数据/压测/授权），这是我们下一步。"

> 禁忌：**不要编数字、不要硬撑、不要说"这个很简单"、不要贬低竞品。**

---

5. 反问评委（结尾可加，显成熟）

- "在您看来，可信 AI 运维落地，最大的障碍是技术还是组织信任？"
- "如果我们优先补真实评测，您建议先攻'检索质量'还是'假成功检测'？"
- "算法创新赛里，您更看重'单点算法新颖'还是'系统级可信整合'？"

---

6. 一页速记卡（打印带上台）

```
一句话：AI 负责思考，人负责授权，系统负责留痕。
三道锁：证据锁 / 风险锁 / 边界锁；一本账：审计账。

混合检索：并行召回 → 各自max归一化 → 0.6向量+0.4BM25 → 合并去重top_k
trust_score：0.5×证据均分 + 0.3×是否被支持 + 0.2×min(证据数/3,1)
四级纠察：Rule(规则) → Evidence(证据) → AI(可插拔) → Verifier(状态核验)
假成功：声称已解决 但 实际resolved=false → REJECTED
风险：CRITICAL禁自动 / HIGH必审批(server.restart·stop·delete·shell.execute)
数字：610 Python 用例 + 671 后端 @Test · 126后端类 · 119表 · 5模块 · 6思考阶段 · 零密钥可复现

被问数据：先认NOT RUN → 亮"诚实评测框架" → 转"系统能力可演示可验证"
被问边界：先承认 → 给路线图(OCR/端侧/多节点/语义版本)
被问竞品：差异在约束，不在模型
```

7. 附录：关键代码索引（评委要核对时，秒回不慌）

主题	文件
混合检索融合	`knowledge/retrieval/hybrid.py`
版本过滤	`knowledge/retrieval/version_filter.py`
可信度评分	`knowledge/evidence/trust_score.py`
证据与引用	`knowledge/evidence/citation.py`
四级纠察流水线	`agent/inspector/pipeline.py`
规则纠察	`agent/inspector/judges/rule_judge.py`
证据纠察	`agent/inspector/judges/evidence_judge.py`
AI 判定（可插拔）	`agent/inspector/judges/ai_judge.py`
状态核验/假成功	`agent/inspector/verifiers/state_verifier.py`
风险分类	`agent/policies/risk.py`
策略引擎	`agent/policies/policy_engine.py`
安全守卫	`agent/safety/guard.py`
人工审批	`agent/safety/approval.py`
SSE 思考流	`ai-service/app/services/chat.py`
RAG 编排	`ai-service/app/services/rag.py`

> 记住：**能被追问到代码文件，是"真做过"最硬的证明。**

册 三

问答圣经 · 速查补充（40 题）

源文件：defense-qa.md

智维Agent 答辩问答圣经（35 题 · 算法创新赛版）

用途：答辩前通读，按类别背"要点"而非背全文；被追问时，用 **"结论先行 + 一句话证据 + 一句路线图"** 结构回应。
核心心法：**先承认边界，再给路线图，比硬撑更得分。**
配套：`defense.md`（陈述稿）、`innovation.md`、`project-plan.md`、`demo-script.md`

一、项目理解 & 定位（5 题）

Q1：用一句话说清楚你做了什么？

A：一个**可信 AI 智能运维平台**——用"版本感知检索 + 证据验证 + 风险策略 + 人工审批 + 全程审计"，把大模型从"会回答问题"变成"可信地完成运维任务"。一句话：**AI 负责思考，人负责授权，系统负责留痕。**

Q2：为什么选"可信运维"这个方向？

A：中小企业"人手少、系统杂、知识散"，最怕两件事——AI 给错建议没人发现、AI 做错动作没人拦得住。通用聊天机器人解决不了这两个"可信"问题，这是真实且未被充分满足的需求。

Q3：你们的用户是谁？场景是什么？

A：10–200 人规模企业的运维负责人/驻场工程师、技术负责人，以及高校实验室/实训机房。核心场景：故障问答定位、版本感知知识检索、受控执行、结果核查与经验沉淀、团队协同。

Q4：和通用的"运维知识库"有什么区别？

A：知识库只解决"查得到"，我们解决"查得对版本 + 答得有证据 + 做得有边界 + 过程能复盘"。知识库是零件，我们是整机闭环。

Q5：项目做到什么完成度？

A：四端全栈可运行（Vue 前端 / Spring Boot 后端 / FastAPI AI 服务 / Python Agent 框架）；前端 `npm test` 494 项全过、后端 126 测试类；默认 mock 零密钥跑通全链路。模型训练与真实基准评测尚未接入（见第六类诚实题）。

二、算法 / 技术深度（8 题）—— *算法创新赛重中之重*

Q6：你们的"算法创新"具体在哪？

A：三点可量化算法设计——① **版本感知混合检索**：BM25+向量并行召回 → `version_filter` 版本过滤 → 词法重排 → 聚合为 `trust_score`；② **四级证据核查（规则→证据→AI→状态核验）（inspector）**：规则+证据+AI 融合判定，降假成功率；③ **风险策略引擎**：基于权限/风险注册表给每步动作打级，高危转人工。

Q7：RAG 为什么不用现成框架（LangChain/LlamaIndex）？

A：现成框架解决"召回"，解决不了"版本过滤 + 证据表达 + 可信度聚合"。我们在 `knowledge/retrieval/version_filter.py` 与 `knowledge/evidence/` 做了场景化扩展，且保留可评测指标。框架是工具，算法是我们自己写的。

Q8：trust_score 怎么算出来的？可解释吗？

A：由检索命中的 Citation（来源/chunk/版本/分数）聚合得到，可逐条溯源到具体知识条目与版本。它是"证据强度"的量化，不是模型黑盒打分——所以可解释、可复核。

Q9：向量嵌入用的什么？效果如何？

A：默认轻量 `hash_embedding`，可一键切换 Sentence Transformers。当前未跑真实基准（见诚实题），但这套接口与评测框架已就绪，接真实模型即可出数。

Q10：Agent 是单次 LLM 调用吗？规划怎么做？

A：不是。planner 把故障拆成**有依赖关系的步骤**（`steps.py`），executor 逐步调用工具，inspector 逐步核查，memory 沉淀长期经验。这是多模块协作，不是 chain 串接。

Q11：你们自研 Agent 框架，图什么？

A：我们需要"策略与审批优先"的执行模型，而不是链式编排。自研 kernel 提供插件注册、依赖解析、事件总线、生命周期，能把**风险判定插在每一步执行之前**——这是通用框架难以做到的。

Q12：四级证据核查（规则→证据→AI→状态核验）里"AI 判定"不会引入新幻觉吗？

A：所以才用三级**融合**而非单靠 AI。规则做硬约束、证据做一致性校验，AI 只做语义级补充判断，最终结论需多路一致。即便 AI 误判，规则与证据层仍能兜底，且全程留痕可追溯。

Q13：多智能体 / 团队 Agent 体现在哪？

A：后端 `TeamAgentRunService` + `DecisionLoopService` 支撑团队协同的智能体运行与决策循环；前端有团队工作区、Agent 运行与审批、通知。单智能体闭环 + 团队级协同是我们 Agent 能力的两个层次。

三、创新点（4 题）

Q14：你的最大创新点是什么？

A：**可信闭环工程化**——把权限、风险、审批、核查、审计做成**可运行的代码路径**，而非文档理念。这是从"AI 对话"到"AI 运维"的关键一跃，且每个创新点都有代码位置与演示路径可验证。

Q15：版本感知检索算创新吗？别人没做过？

A：版本过滤在文档管理里有，但把"版本"作为**检索的一等公民**并融入 RAG 可信度聚合，在通用运维场景里是实质性差异。中小企业最常见的故障正是"查到旧版本"，这是真实痛点驱动的创新，不是为创新而创新。

Q16：透明推理（思考可视化）是噱头吗？

A：不是。SSE 输出 `thinking/token/done` 三类真实事件，思考阶段由后端真实计算（含命中证据条数、图像证据条数），前端只是渲染。它直接提升运维人员对 AI 的**信任度**——而信任，正是"可信运维"的命脉。

Q17：你们有多少个创新点？会不会散？

A：文档列了七个（可信闭环、版本感知检索、透明推理、多模态隐私边界、插件化工具、可切换模型网关、前端工程化），但答辩只深讲**最核心的 3 个**，其余一句带过。评委要的是"深"，不是"多"。

四、诚实边界 / 评测（5 题）—— *高频陷阱题，必须练熟*

Q18：你的准确率 / 效果数据是多少？

A：坦诚讲——**当前未接入真实模型与标注数据集，没有编造的基准数字。**但我们搭建了诚实评测框架：消融实验 + 基线对比链 + leakage 检测 + 指标契约（任务成功率/根因准确率/证据准确率/Inspector 精确率召回率/幻觉率/假成功率/安全违规率）。无真实观测数据前系统强制 `NOT RUN`。**一个知道"哪些结论还不能下"的系统，比什么都敢编的系统更可信。**

Q19：没数据凭什么说你好？

A：好分为两层——**系统能力**已可演示可验证（闭环、检索、审批、审计都在跑）；**模型效果**需要真实数据，我们如实标注未跑。竞赛看的是算法设计与工程实现，这两层我们拿得出手。

Q20：你的消融实验做了吗？

A：实验设计已完成（RAG 六组、Inspector 五组），但**尚未执行**——因为缺真实模型/数据集/evaluator 三件套，门禁不允许空跑。设计本身已是严谨工作的体现，执行是下一步。

Q21：为什么用 mock 模型？是不是根本没接真模型？

A：mock 是为**零密钥可复现**，不是能力上限。`model_gateway` 支持 Ollama 与 OpenAI 兼容接口，运行期切换；后端还管模型

连接、路由、Token 预算与用量。评委可以自己拉下来接真模型跑。

Q22：你说"可信"，敢不敢现场把高危动作真执行?

A：**故意不敢**——这正是设计意图。高危动作必须人工审批，批准后才在沙箱执行。现场演示我**会**展示一次"被拦截→审批→执行→审计"的完整链路，但绝不会绕过审批直接动生产。安全不是演示出来的，是架构逼出来的。

五、工程 / 可复现（4 题）

Q23：代码量 / 测试情况?

A：前端 `npm test` 494 项全过（99 个测试文件）、`tsc --noEmit` 通过、双入口构建通过；后端 126 个 JUnit 测试类（H2 MySQL 兼容模式跑真实迁移）；MySQL 8 / 119 张表、迁移幂等、启动自动补齐。质量门禁命令全在 `project-plan.md`。

Q24：前端为什么"部分 TypeScript"?

A：有意的取舍。共享逻辑层（api/domain/utils/types 共 48 模块）迁 TS 拿类型收益；组件层保持 JS 控改造成本。`tsc --noEmit` 已纳入门禁，不是偷懒。

Q25：119 张表是不是过度设计?

A：覆盖账户权限、审计、团队协作、知识、模型网关、哨兵监控、远程运维等**完整域模型**。迁移脚本幂等，局部缺失只影响对应模块，不影响整体。

Q26：性能如何? 大屏/3D 会不会卡?

A：分包针对性优化：zrender 从 echarts 拆出（510kB→346kB），Three.js 懒加载，管理员层独立产物，图表面板按需加载。渲染侧做了按需加载。

六、安全 / 隐私（4 题）

Q27：数据安全怎么保证?

A：BCrypt(12) 哈希、CSRF、会话固定防护、管理员 MFA + 恢复码、AES-GCM 密钥加密、附件下载 `nosniff` + CSP sandbox、关键事件结构化审计留痕。

Q28：多模态是"假多模态"吗?

A：诚实回答——当前图像是**文件名级**参与链路，图片内容不上传（刻意隐私取舍）；语音在浏览器内识别、音频不出端。仓库已有 OCR 模块，路线图是在用户授权后接内容级理解。**我们不把"传文件名"包装成"看懂图片"。**

Q29：模型密钥怎么管?

A：密钥经环境变量注入，AES-GCM 加密存储于后端，绝不入库、绝不明文。`.env` 已 gitignore。

Q30：AI 自动执行会不会很危险?

A：三重约束——权限注册表、风险策略引擎、沙箱；高危必须人工审批；执行后 inspector 核查 + 审计。危险动作在架构层就被"卡死"。

七、商业 / 落地（3 题）

Q31：商业闭环是什么?

A：SaaS 订阅（按席位）+ 私有化授权 + 知识库/插件增值模块 + 模型用量分成。目标客户 10–200 人规模企业。

Q32：中小企业凭什么买单?

A：默认 mock 可离线演示、私有化可自托管数据、降低对"老师傅"个人经验的依赖。成本远低于养一支完整运维团队。

Q33：下一步计划?

A：图像内容级理解（授权后）、端侧推理（Transformers.js/WebGPU）、多节点会话与 MFA 强化、团队协同深化。每条都对应现在已留好的扩展点。

八、对比 / 竞品（3 题）

Q34：这和 ChatGPT + 提示词有什么区别?

A：差异**不在模型，在约束。**我们有版本感知检索、证据验证与可信度评分、风险分级与人工审批、审计留痕与经验回流。模型可换，这套约束不换。ChatGPT 给答案，我们给"可负责任的运维动作"。

Q35：和 AIOps 平台（如传统监控告警）比?

A：传统 AIOps 多停留在规则/阈值告警，有日志无推理、有自动化缺审批。我们补上了"证据可信度 + 风险审批 + 透明推理 + 审计沉淀"四块短板，且是 AI Native 的对话式入口。

九、压力 / 陷阱题（附加 • 练心态）

Q36："我觉得你这项目没什么技术含量。"

A：理解您的视角。我们的技术含量不在"训练了一个大模型"，而在**把可信约束工程化**——版本感知检索算法、四级证据核查（规则→证据→AI→状态核验）、风险策略引擎、可复现评测框架，每一处都是针对"AI 运维可信"这一真实难题的算法与系统设计。欢迎就任意模块深入切磋。

Q37："你的创新点别人都能做，凭什么获奖？"

A：单点别人或许能做，但把"版本过滤 + 证据聚合 + 风险审批 + 透明推理 + 全程审计"**串成一个可运行、可演示、可复现的闭环**，且每个点都有代码与演示路径，这种**系统性整合与工程纪律**本身即是壁垒。

Q38：现场演示失败怎么办?

A：预案——① 60 秒备用录屏；② 直接展示 `technical-document.html` 链路图；③ 口述 Demo 主线并点开对应代码位置。不慌、不重来、不长时间停顿。

Q39：团队只有 1–3 人怎么保证完成度?

A：按模块切分（前端交互 / 后端数据 / AI 算法），用测试与构建门禁防单人阻塞，文档与脚本保证可交接。610 个 Python 用例 + 671 个后端 @Test就是"不会因为某个人请假而崩"的保证。

Q40：如果模型答错了怎么办?

A：三层兜底——提示词约束"无证据不下确定结论"；证据链让用户可核对；高风险动作必须人工审批。答错不会直接变成生产事故。

速记卡（候场最后看一眼）

- 一句话：**AI 负责思考，人负责授权，系统负责留痕。**
- 三道锁：证据锁 / 风险锁 / 边界锁；一本账：审计账。
- 算法三板斧：版本感知混合检索 → trust_score；四级证据核查（规则→证据→AI→状态核验）；风险策略引擎。
- 数字武器：610 个 Python 用例 + 671 个后端 @Test / 126 后端类 / 119 表 / 5 模块 / 6 思考阶段 / 零密钥可复现。
- 被问数据：先承认 NOT RUN，再亮"诚实评测框架"，转回"系统能力可演示可验证"。
- 被问边界：先承认，再给路线图（OCR/端侧/多节点）。

册 四

答辩补强 • 技术追问 / 产品优势 / 未来前景

源文件：defense-advanced.md

答辩补强篇（临场技术追问 • 产品优势 • 未来前景）

配合 `defense.md`（陈述稿）、`defense-master.md`（70 题主问答）使用。
本册专攻三件评委最爱追问、也最容易翻车的事：**技术到底怎么实现、产品凭什么赢、未来凭什么信。**
原则不变：**结论先行 → 一句证据 → 一句边界/路线图。先承认，再超越。**

0. 先用 30 秒记住"12 张技术底牌"（自曝边界，反而最稳）

评委只要真读代码，这 12 个点一定藏不住。**提前知道、敢承认、给路线**，你就从"被问倒"变成"显得通透"。

#	真实情况（别硬吹）	一句话应对
1	BM25 中文**没分词**，连续中文当一个整词（正则 `[\u4e00-\u9fff]+`）	"规则分词基线，零依赖可复现；生产接 jieba/字符 n-gram，中文语义交给向量路。"
2	向量库是**内存线性扫描 + 余弦**，非 FAISS	"小语料足够；规模化换 FAISS / pgvector，接口已抽象。"
3	默认嵌入是 **hash_embedding**，不是语义模型	"演示默认，可切 Sentence Transformers；它不代表能力上限。"
4	Planner 是**规则基线**（关键词），不是 LLM	"确定性基线先跑通、可测；LLM Planner 是插件化替换项。"
5	Executor **没有自动重试/幂等键/回滚**	"失败即上报 inspector 触发 Re-plan；幂等与回滚在路线图。"
6	AI Judge **默认 passthrough**（不绑模型）	"可插拔钩子，默认不阻塞；生产注入评审模型。"
7	证据一致性是**词面重合**启发式	"确定性、不被'自信错误'骗；语义升级走 NLI/模型。"
8	版本过滤是**精确匹配**	"精度换召回，宁缺毋滥；升级语义化版本区间。"
9	**没有真实 benchmark**，全部 `NOT RUN`	"搭了诚实评测框架，无数据不造数。"
10	图像**只传文件名**，内容不出端	"刻意隐私取舍；OCR/多模态是授权后路线。"
11	119 张表**偏重**	"完整域模型 + 幂等迁移；可模块化裁剪核心子集。"
12	**未做系统压测**、单机部署	"分包与懒加载做过优化；压测与水平扩展在计划内。"

记住：**能主动说出第 1、3、4、5、6 条的人，评委会觉得你是真懂，而不是背稿。**

1. 临场技术追问 60 题（深挖级）

每题结构：**评委想挖什么 → 通透过答 → 证据锚点 → 别踩的坑。**打 ⚠️ 的是"真短板"，必须按第 0 节话术答。

A. 检索与算法（14 题）

A1 ⚠️ BM25 中文怎么分词的?

想挖：你有没有真处理中文。答：当前 `tokenize()` 用正则，**连续中文整体当一个 token，未分词**——所以长中文句召回弱，但英文标识符/错误码（`max_connections`、`502`）召回强，中文语义由向量路补。升级：jieba/HanLP 或字符 2-gram。证据：`knowledge/retrieval/bm25.py`。

A2 BM25 参数?

答：`k1=1.5, b=0.75`（经典默认），IDF 用 `log(1+(N-df+0.5)/(df+0.5))`。可调，未用数据调优——诚实说"经验默认值"。

A3 向量维度多少？相似度怎么算?

答：维度随嵌入后端而定；相似度用**余弦**（`vector_store.py` 手写实现，含零向量保护）。默认 hash 嵌入，可切 Sentence Transformers。

A4 ⚠️ 内存线性扫描，规模大了怎么办?

答：当前 `InMemoryVectorStore` 是 O(N) 线性，适合小语料/演示；规模化换 **FAISS / pgvector / Milvus**，`search()` 接口已隔离，替换不影响上层。诚实承认"没做大规模"。

A5 两路分数怎么融合？为什么 0.6/0.4?

答：各自按本路 max 归一化，再加权（向量 0.6、BM25 0.4）。权重是**可配置初值**，理由：语义是主诉求，精确词是补充。未用数据拟合——诚实。

A6 🔥 为什么不用 RRF?

答：RRF 秩融合鲁棒、免调参；我们用归一化分加权，可解释、可按场景调权。两者都合理，RRF 在消融计划里。**别贬低 RRF。**

A7 只用 max 归一化，离群值会压扁其他分?

答：会。max 归一化对离群敏感，min-max 又依赖最小值；当前选简单稳定版，可切。承认局限 + 给替代。

A8 版本过滤怎么做的？会不会误杀?

答：`version` 为空或 `latest` 不过滤，否则**精确匹配** `chunk.version`（召回阶段已下推）。同义版本号会误杀 → **精度换召回**，升级语义化版本区间。

A9 重排用的什么？为什么不用 cross-encoder?

答：词法重排（`lexical_reranker.py`）做确定性收敛；cross-encoder 效果好但重、需模型，**当前为可复现选了轻量**，是路线图。

A10 知识库怎么更新/增量索引?

答：文档解析→分块→嵌入→入索引；当前偏全量重建，**增量索引是待补**。诚实。

A11 多版本文档冲突怎么处理?

答：检索按目标版本过滤 + 证据层做版本一致性检查（不匹配给 WARNING）；这是"版本为一等公民"的落点。

A12 RAG 幻觉怎么压?

答：三层——提示词"无证据不下确定结论"；证据链让用户可核对；inspector 证据层判 SUPPORTED/UNSUPPORTED。**承认不能完全消除。**

A13 top_k 怎么定?

答：默认 5，可配；两路各取 top_k 再融合。未做系统性调参。

A14 你的检索"算法创新"能不能形式化?

答：能——融合是归一化加权函数；可信度是 `0.5·mean(score)+0.3·support+0.2·coverage`；纠察是"多信号 → 状态机"。**都可写成公式、可消融**，这是"算法"而非"调包"。

B. Agent 与执行（12 题）

B1 ⚠️ Planner 是 LLM 还是规则?

答：**规则基线**（关键词：含"nginx+状态"→查状态；含"重启/restart"→重启且 `requires_approval=True`；否则查知识库）。好处：确定性、可测、零成本。LLM Planner 是插件化替换项。证据：`agent/planner/planner.py`。**别说成"LLM 自主规划"。**

B2 规划失败/多步依赖怎么办?

答：当前规划是单步为主、带 `requires_approval` 标记；多步依赖结构与 Re-plan 在路线图。诚实。

B3 ⚠️ 工具执行失败会重试吗？有回滚吗?

答：**没有自动重试/幂等键/回滚。**Executor = 策略校验 → dry_run → 执行 → 审计；异常返回失败并记审计，交由 inspector 触发 Re-plan。幂等/回滚是明确的下一步。证据：`agent/executor/executor.py`。

B4 有 dry_run 吗?

答：有。`context.dry_run=True` 时只返回"未实际执行"+参数，不碰生产。**演示安全动作时可先 dry_run。**

B5 循环/滥用怎么防?

答：RuleJudge 统计单工具调用次数，**>10 次判 LOOP（HIGH）**。阈值可配。

B6 权限模型?

答：RBAC（角色/部门/岗位）+ `PermissionChecker` 允许角色集（admin/operator）；策略引擎 + SafetyGuard 双闸。

B7 Agent 内核有什么?

答：五类注册表（plugin/skill/tool/model/permission/risk）+ 事件总线 + 生命周期（MOUNTING→MOUNTED→FAILED→STOPPED）。证据：`agent/kernel/runtime/runtime.py`。

B8 插件怎么装?

答：`AgentPlugin` 契约注册，`mount/unmount` 生命周期；内置 Docker/K8s/MySQL/Nginx/Redis；MCP 适配器接外部。

B9 会话隔离/并发?

答：会话按 conversation_id 隔离；存储支持 Redis/MySQL/PostgreSQL/SQLite/文件五种。**高并发未压测**，诚实。

B10 上下文太长怎么办?

答：目前靠检索注入少量证据（top_k 小）+ 短期记忆；**长上下文压缩/滑窗是待补。**

B11 记忆怎么写入/遗忘?

答：短期上下文 + 长期 `operator_memory`；写入以处置结论为主；**遗忘/衰减策略未细化**，诚实。

B12 Agent 会不会自己"发明"工具?

答：不会。工具必须先在注册表注册，Executor 只执行**已注册**工具；未知工具直接返回"工具不存在"。

C. 证据与可信（10 题）

C1 trust_score 公式?

答：`clamp(0.5·mean(证据分)+0.3·[被支持]+0.2·min(证据数/3,1), 0,1)`，4 位小数。可逐项拆解。

C2 权重 0.5/0.3/0.2 凭什么?

答：经验初值（检索准 > 结论有据 > 证据量），**待消融标定。**诚实。

C3 ⚠️ 证据一致性用词重叠，太粗?

答：是确定性启发式（重合率 ≥0.5 支持 / ≥0.2 部分 / 否则不支持）。好处：**不被"自信的错误答案"骗过**；语义升级走 NLI/模型（L3）。

C4 四级纠察是哪四级?

答：Rule（规则）→ Evidence（证据）→ AI（可插拔）→ State Verifier（状态核验）。聚合：false_success→REJECTED；HIGH/CRITICAL→VIOLATION；其余有 issue→WARN；否则 PASS。

C5 假成功怎么判?

答：`StateVerifier` 拿"声称已解决/恢复"对比 `actual_state.resolved`；声称解决但实际 `false` → **false_success** → REJECTED + Re-plan。

C6 ⚠️ AI Judge 默认是空的，那"AI 判定"是不是摆设?

答：是**可插拔钩子**，默认 passthrough 保证"无模型也不阻塞安全链路"；生产注入评审模型。**真正的确定性安全来自 Rule+Evidence+Verifier，不依赖 LLM**——这反而是优点。

C7 审计日志能防篡改吗?

答：当前是结构化留痕 + 数据库存储；**哈希链/WORM 未做**，生产可接哈希链或不可变存储。诚实。

C8 怎么证明不是"自评自嗨"?

答：评测契约要求统一分组集、留 artifact hash、缺失记 `null` 不记 0、零分母不报 0；无观测强制 `NOT RUN`。**这是防自嗨的机制。**

C9 用户凭什么信 trust_score?

答：它是**可溯源**的（每条 Citation 带来源/版本/分数），不是模型黑盒；但权重未标定，所以定位是"证据强度参考"，不是"绝对正确率"。

C10 证据不足时系统怎么表现?

答：证据层给 WARNING/UNSUPPORTED；提示词要求"明确说明无法确认"；不会硬编一个确定结论。

D. 安全与隐私（8 题）

D1 沙箱是真隔离还是逻辑约束?

答：当前是**策略/权限层约束**（sandbox policy：文件/进程/网络规则）+ 权限注册表；**非容器级强隔离**。生产可叠 ns/cgroup/seccomp。诚实。

D2 密钥怎么存?

答：环境变量注入 + AES-GCM 加密存后端，绝不入库/明文；`.env` gitignore。**未接 KMS/HSM**，生产可接。

D3 MFA、CSRF 怎么做的?

答：管理员 MFA + 恢复码；CSRF Token 校验写操作；SSE 经 Vite 代理去前缀，跨端口同源由 `vite.shared.mjs` 统一，防会话/CSRF 分裂。

D4 越权怎么防?

答：RBAC + 允许角色集 + 策略引擎；管理端敏感操作二次确认 + 审计。

D5 符合哪些合规?

答：隐私最小必要（音频/图片不出端）、密钥加密、审计留痕，对齐 **PIPL 最小必要**思路；**等保测评未做**，诚实。

D6 多模态隐私边界?

答：语音浏览器内识别（音频不出端）、图像仅文件名（内容不出端）、附件隔离不参与模型分析。

D7 高危动作被拦截后能强执行吗?

答：CRITICAL 直接禁；HIGH 必须审批；**没有"绕过"开关**（这是设计约束，不是缺陷）。

D8 如果审批人点错了批准?

答：执行前有 dry_run/风险提示；执行后有 inspector 核查 + 审计可回溯；**误批仍是风险**，靠流程+审计兜底。

E. 工程与性能（8 题）

E1 610 个 Python 用例 + 671 个后端 @Test是不是凑数?

答：覆盖契约、安全不变量、领域逻辑，是"防单人阻塞"的交接保证，不是纯断言堆砌。

E2 有压测/QPS 吗?

答：**没有**。分包做过优化（echarts 510→346kB、Three 懒加载），但**未做系统压测**；水平扩展在计划内。诚实。

E3 119 张表会不会太重?

答：完整域模型（账户/审计/团队/知识/模型/哨兵/远程运维）；迁移幂等；**可模块化裁剪核心子集**。

E4 服务间一致性（AI/知识/后端）?

答：AI→知识走 HTTP，知识不可用时**优雅降级为空结果**（`rag.py` 的 `_empty_result`），不阻塞对话；**无分布式事务**，因为场景不强依赖。

E5 单点故障?

答：单机部署，**无高可用**；生产可多实例 + 网关。诚实。

E6 前端首屏/包体积?

答：zrender 独立拆分、Three 懒加载、管理员层独立产物、图表面板按需加载。

E7 数据库迁移怎么保证不炸?

答：迁移脚本**幂等**，启动自动补齐；H2（MySQL 兼容）跑真实迁移做测试。

E8 为什么四端而不单体?

答：职责分层——Spring 管权限事务、FastAPI 管 AI 流式、知识服务独立可评测算、Python Agent 管执行。**代价是运维复杂度**，用 docker-compose 收敛。诚实承认权衡。

F. 模型与评测（8 题）

F1 为什么不自己训模型?

答：竞赛周期内训不出可靠领域模型；价值在**约束与工程**，模型可换。自训/微调是路线图。

F2 SFT/LoRA 计划?

答：有训练管线骨架（`training/`），但**未执行**（缺真实模型/数据/依赖）；属于下一步。

F3 评测指标有哪些?

答：任务成功率、根因准确率、证据准确率、工具/风险/审批准确率、验证成功率、Inspector P/R、幻觉率、假成功率、安全违规率、延迟、Token 成本。

F4 消融怎么设计?

答：RAG 六组（Vector→BM25→Hybrid→+Rerank→+Version→+Evidence）、Inspector 五组（Agent→+Rule→+Evidence→+AI→+Verification）。

F5 数据泄漏怎么防?

答：按事件族/根因/环境/版本/场景**分组切分**，写 `leakage_report.json` 才算接受。

F6 为什么用 mock?

答：零密钥可复现，不是能力上限；`model_gateway` 可切 Ollama/OpenAI 兼容。

F7 评测没跑，竞赛怎么算?

答：如实标注 `NOT RUN`；评的是算法设计与工程实现，这两层可演示可验证。

F8 ⚠️ 你的算法"可证伪"吗?

答：可——融合/可信度是公式，纠察是状态机，消融可跑；**设计上允许被数据推翻**，这是科学态度。

G. 多模态（4 题）

G1 图像为什么只传文件名?

答：刻意隐私取舍（内容不出端）；OCR 模块已在，授权后接内容级+多模态大模型。

G2 语音识别准吗？方言?

答：Web Speech API 浏览器内置识别，**准确率/方言依赖浏览器**；不支持时入口自动隐藏（渐进增强）。

G3 多模态算不算"假多模态"?

答：文件名级参与是**如实边界**，不包装成"看懂图片"；这是可信的一部分。

G4 接视觉模型要改架构吗?

答：不用大改，`ChatRequest.images` 已是扩展点，换内容级解析即可。

H. 架构与选型（6 题）

H1 和微服务比，你这算微服务吗?

答：算**轻量多服务**（前端/后端/AI/知识/Agent），docker-compose 编排；不是完整 service mesh。

H2 为什么后端用 Java，AI 用 Python?

答：Java 成熟权限/审计/事务；Python AI 生态与异步流式。职责分层。

H3 为什么自研 Agent 内核?

答：要"策略与审批优先"，把风险判定插在**每步执行前**；通用链式编排做不到执行前拦截。

H4 能上云/容器吗?

答：docker-compose 一键起；可容器化上云；生产需配 TLS/网关/多实例。

H5 数据库为什么 MySQL 不用 PG?

答：团队熟悉 + 生态成熟；会话存储已支持 PG 等多后端，切换成本低。

H6 前端为什么部分 TS?

答：共享逻辑层（48 模块）迁 TS 拿类型收益，组件层留 JS 控成本；`tsc --noEmit` 进门禁。有意取舍。

---

2. 产品 / 项目优势（评委问"凭什么"时用）

2.1 一页差异化矩阵

维度	ChatGPT/通用问答	Coze/Dify 编排	传统 AIOps	本产品 智维Agent
回答依据	模型记忆	工具返回，缺证据链	规则/阈值	版本感知混合检索 + 证据验证
可信度	无	无	告警等级	Citation + trust_score 可溯源
知识版本	不分	不分	弱	版本一等公民（过滤+一致性检查）
执行权限	不执行	默认偏大	部分自动化缺审批	策略引擎 + 人工审批 + 沙箱
过程透明	黑盒	有轨迹无用户视图	有日志无推理	SSE 分阶段思考可视化
审计沉淀	无	有限	有日志	审计留痕 + 经验回流
多模态隐私	直接上传	直接上传	不涉及	分层边界（音频/图像/附件）

2.2 五大硬优势（背这五句）

1. **可信闭环工程化**——权限/风险/审批/核查/审计是**可运行代码**，不是 PPT 理念。
2. **版本感知检索**——解决"知识正确但版本不对"的行业通病，版本做进召回。
3. **透明推理**——用户看得见检索了什么、命中多少证据、判了什么风险，直接建立信任。
4. **隐私边界分层**——"能用"和"该传"分开，合规友好。
5. **零密钥可复现 + 模型可切换**——评委能自己跑，企业能接自有模型，兼顾演示与生产。

2.3 真正的壁垒（评委最爱问"护城河"）

> **不是模型，是"约束工程 + 领域知识资产 + 插件生态 + 评测框架"的系统性整合。**
> 模型谁都能换；把"可信"做成可拦截、可追溯、可复现的闭环，才是壁垒。

2.4 成本 / 部署优势

- **演示零成本**：mock 零密钥跑通全链路。
- **落地可控**：私有化自托管数据；核心子集可裁剪，降低 119 表的部署门槛。
- **模型可换**：Ollama 本地省成本，OpenAI 兼容按需用。

---

3. 未来前景（评委问"能走多远"时用）

3.1 产品路线图

阶段	重点
近期（1–3 月）	接真实模型跑通评测；中文分词升级（jieba/n-gram）；向量库换 FAISS/pgvector；executor 幂等/重试
中期（3–6 月）	LLM Planner；多智能体团队协同；OCR/多模态；压测与水平扩展；KMS 接入
远期（6–12 月）	端侧推理（Transformers.js/WebGPU）；插件市场/知识市场；SaaS + 私有化双形态；等保/PIPL 合规

3.2 技术演进三条主线

1. **可信深化**：哈希链审计、形式化风险策略、验证驱动 Agent。
2. **能力扩展**：多模态内容级、多智能体、端侧。
3. **工程规模化**：向量库、压测、多实例、可观测性（Prometheus/Grafana/Loki 已在 infra）。

3.3 市场与商业化（讲逻辑，不吹数字）

- **目标客户**：10–200 人企业、高校实验室/实训机房。
- **收费**：SaaS 按席位 + 私有化授权 + 知识库/插件增值 + 模型用量分成。
- **渠道**：高校/实训切入（同源需求）→ 中小企业口碑 → 行业方案商。
- **TAM 逻辑**（可讲推理，不报虚构数）：中小企业数量庞大 × 运维人力成本高 × 现有 AIOps 不普惠 = 空白带。

3.4 学术价值（算法创新赛加分）

- 可信 RAG 的**版本感知检索 + 可信度聚合**；
- **假成功检测**（verification-driven agent）；
- **验证驱动的 Agent 安全闭环**；
- 可沉淀 **OpsBench 类评测基准**（技术文档已提及样例数据集）。

3.5 风险与应对（敢说风险=成熟）

风险	应对
模型成本/不可用	mock 兜底 + Ollama 本地 + 路由与预算
隐私合规	分层边界 + 最小必要 + 私有化
大厂竞争	垂直可信 + 私有化 + 领域资产，不拼通用
落地信任	人工审批 + 审计 + 可复现演示

---

4. 临场刁钻题（新增 20 题，练心态）

Z1 "给你 100 万怎么花？"

答：40% 真实评测与数据（把 NOT RUN 跑成数）、30% 检索/向量工程化、20% 多模态与端侧、10% 合规与压测。**先补最硬的短板（数据）。**

Z2 "能用在哪三个行业？"

答：高校实验室/实训机房（同源）、中小企业 IT 运维、MSP/IT 外包服务商（多租户运维）。

Z3 "大厂明天做一个一样的怎么办？"

答：大厂做通用平台，我们做**垂直可信 + 私有化 + 轻量可复现**；它们不会为 50 人企业做深度定制，这是我们的空间。

Z4 "你给自己打几分，扣在哪？"

答：设计/工程 8 分，**真实评测 4 分**（还没跑），多模态深度 5 分。扣分点我清楚，也知道怎么补。

Z5 "一句话说服我投资。"

答：企业不敢把生产交给只会回答的 AI；我们让它变成**敢托付的 AI 同事**——结论有证据、动作有边界、过程有留痕。

Z6 "只能保留一个功能，留哪个？"

答：**风险审批闸门**。没有它，AI 越强越危险；有它，才谈得上"可信"。

Z7 "你项目最大的风险是什么？"

答：真实数据闭环没跑通——评测算不出来，一切效果都是设计推断。这是最硬短板，也是第一步要补的。

Z8 "怎么证明不是 AI 帮你写的？"

答：架构/算法/验收由人把关，关键路径我讲得出为什么这么设计、边界在哪、代码在哪。**能追到实现细节，就是证明。**

Z9 "如果模型免费了，你还剩什么？"

答：约束闭环、证据链、审批审计、版本治理、领域知识——**模型免费，信任不免费。**

Z10 "用户凭什么信你的 trust_score？"

答：可溯源（来源/版本/分数可拆）；且定位是"证据强度参考"不是"绝对正确率"，诚实。

Z11 "为什么不用现成大模型 Agent 平台？"

答：平台给你编排，给不了"执行前风险闸门 + 假成功检测 + 版本感知检索"；这些是垂直可信的关键。

Z12 "你的系统能替代运维吗？"

答：不替代，**增强**。AI 负责思考与执行建议，人负责授权，系统负责留痕——是"AI 同事"不是"AI 替身"。

Z13 "为什么评委要给你奖，而不是给一个更炫的？"

答：炫的很多，**敢把"可信"做成可运行闭环**的少；我们每个创新点都有代码与演示路径，经得起追问。

Z14 "如果现场断网/模型挂了？"

答：mock 本地兜底，全链路不依赖外网；演示有 60 秒备用录屏。

Z15 "你做这个花了多久？团队几个人？"

答：据实说；强调模块化与门禁让"小团队也能交付完整系统"。

Z16 "你觉得 AI 运维的终局是什么？"

答：从"辅助回答"到"可信代理"——关键不是更聪明，而是**可授权、可拦截、可追溯**。我们押"信任基础设施"。

Z17 "为什么不用更小的模型/端侧？"

答：端侧是路线图（隐私/成本/离线）；当前先保证可信闭环与可复现。

Z18 "你的项目有什么社会价值？"

答：把大厂级运维能力普惠给中小企业与高校，降低对少数"老师傅"的依赖，缓解运维人才缺口。

Z19 "如果只能改一个技术点，改哪个？"

答：**中文分词**——它直接决定 BM25 召回质量，是当前最影响效果的单点。

Z20 "用三个词总结你的项目。"

答：**可信、可控、可复盘。**（或：证据、边界、留痕。）

5. 一页"技术速答卡"（候场最后一眼）

【12底牌·敢承认】

中文未分词 / 向量内存线性 / hash嵌入 / Planner规则 / 无重试回滚 / AIJudge可插拔

证据=词重叠 / 版本精确匹配 / 无benchmark / 图像仅文件名 / 119表偏重 / 未压测

【三张公式】

融合：0.6×向量maxnorm + 0.4×BM25maxnorm

可信：0.5×证据均分 + 0.3×被支持 + 0.2×min(n/3,1)

纠察：Rule→Evidence→AI→Verifier；假成功→REJECTED

【三句优势】

可信闭环工程化 / 版本感知检索 / 透明推理

壁垒 = 约束工程 + 领域资产 + 插件生态 + 评测框架（不是模型）

【三层未来】

近期：真实评测·分词·向量库·幂等

中期：LLM Planner·多智能体·多模态·压测

远期：端侧·插件市场·SaaS+私有化·合规

【万能救场】

接住 → 给已知 → 划边界+路线。绝不编数字、不硬撑、不贬竞品。

智维Agent · 一页速记卡（可裁下带上台）

AI 负责思考，人负责授权，系统负责留痕。

三道锁：证据锁 / 风险锁 / 边界锁；一本账：审计账。

混合检索：并行召回 → 各自 max 归一化 → 0.6 向量 + 0.4 BM25 → 合并去重 top_k

trust_score：0.5×证据均分 + 0.3×是否被支持 + 0.2×min(证据数/3, 1)

四级纠察：Rule(规则) → Evidence(证据) → AI(可插拔) → Verifier(状态核验)

假成功：声称已解决 但 实际 resolved=false → REJECTED

风险：CRITICAL 禁自动 / HIGH 必审批（server.restart·stop·delete·shell.execute）

数字：610 个 Python 用例 + 671 个后端 @Test · 119 张表 · 5 模块 · 6 思考阶段 · 零密钥可复现

被问数据：先认 NOT RUN → 亮"诚实评测框架" → 转"系统能力可演示可验证"

被问边界：先承认 → 给路线图（OCR / 端侧 / 多节点 / 语义版本）

被问竞品：差异在约束，不在模型